

M1 MMS : Simulation Numérique

TP1: 1D piecewise polynomial approximations

In all these exercises we focus on approximations of a function $f \in L_2(\Omega)$. Ω is a 1D domain.

Theoretical background (reminder)

Spatial domain $\Omega = [a; b]$ is discretized, using N non overlapping elements $\mathcal{K}_i$ for $i = 0, \dots, N-1$.

The functionnal space P_k continuous is defined as follows:

$$P_k = \{v : \Omega \rightarrow \mathbb{R} : v \in C^0(\Omega), v|_{\mathcal{K}_i} \in \mathbb{R}_k[X], i = 0, \dots, N-1\}$$

For simplicity, in these exercises, we consider N equally sized elements with equally spaced discretizations such that: $a = x_0 < x_1 < \dots < x_n = b$. where $n = kN$ and $x_j = a + j(b-a)/n$ for $j = 0, \dots, n$, therefore $\mathcal{K}_i = [x_{ki}; x_{ki+k}]$ for $i = 0, \dots, N-1$.

Each elements $\mathcal{K}_i$ admits $k+1$ degree of freedom, also called nodes.

A common choice for a basis for P_k is Lagrange basis on the nodes. The basis is denoted $\{\phi_i\}_{i=0, \dots, n}$.

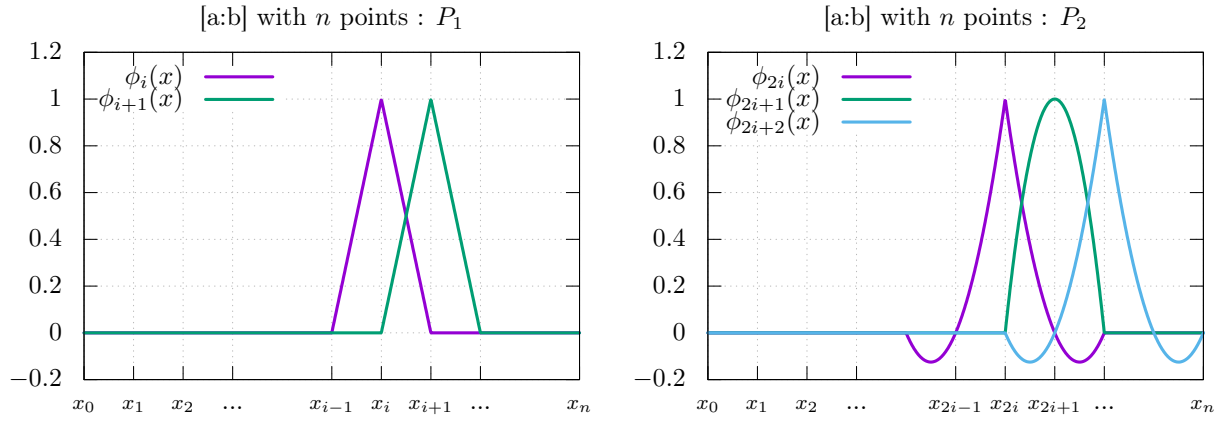
For instance, on P_1 :

$$\phi_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}} & \text{if } x \in [x_{i-1}, x_i] \cap \Omega \\ \frac{x_{i+1} - x}{x_{i+1} - x_i} & \text{if } x \in [x_i, x_{i+1}] \cap \Omega \\ 0 & \text{otherwise} \end{cases} \text{ for } i = 0, \dots, n$$

For instance, on P_2 (n must be even):

$$\phi_{2i}(x) = \begin{cases} \frac{(x - x_{2i-1})(x - x_{2i-2})}{(x_{2i} - x_{2i-1})(x_{2i} - x_{2i-2})} & \text{if } x \in [x_{2i-2}, x_{2i}] \cap \Omega \\ \frac{(x - x_{2i+1})(x - x_{2i+2})}{(x_{2i} - x_{2i+1})(x_{2i} - x_{2i+2})} & \text{if } x \in [x_{2i}, x_{2i+2}] \cap \Omega \\ 0 & \text{otherwise} \end{cases} \text{ for } i = 0, \dots, n/2$$

$$\phi_{2i+1}(x) = \begin{cases} \frac{(x - x_{2i})(x - x_{2i+2})}{(x_{2i+1} - x_{2i})(x_{2i+1} - x_{2i+2})} & \text{si } x \in [x_{2i}, x_{2i+2}] \cap \Omega \\ 0 & \text{otherwise} \end{cases} \text{ for } i = 0, \dots, n/2 - 1$$



Remark

In all the exercises, we will use two meshes : one defining the discretization of the domain $(x_i)_{i=0,\dots,n}$ (used for basis definition, and latter for solution computation), and one for visualisation purposes with 1000 points. This finer mesh will help to observe the abstract representation of function approximation.

The fine mesh, is used only in these exercices to help visualizing the functions, it is never used in practice, nodal evaluations of functions will be enough to plot solutions.

Part 1 : Basis functions

The purpose of this part is to draw all the basis functions for P_1 and P_2 (one degree of approximation per figure).

The functions will be evaluated on a 1000 points mesh but defined on a mesh with only a few points ($n = 6$ nodes for instance).

Successive version of the code will implement small optimisations regarding properties of basis functions together with `numpy` array operations.

Part 2 : Interpolation

We denote $I_n(f) = \sum_{i=0}^n c_i \phi_i$ the approximation of f by interpolation where $c_i = f(x_i)$. We note $C = (c_i)$ the vector of coordinates of $I_n(f)$ in the basis.

Exercise 1 : P_1 approximation

1. Plot the points $(x_i; c_i)$. These are the coordinates of $I_n(f)$ in the ϕ_i basis. Add a plot of evaluations of f on a fine mesh. This would be an acceptable representation of the function f itself.

Indications: use a `scatter` style for plotting the points $(x_i; c_i)$.

2. Add to the figure, a plot of evaluations of $\sum_{i=0}^n c_i \phi_i$ on the fine mesh, using your functions from previous part for evaluations of ϕ_i .

Indications:

- The function for evaluating $\sum_{i=0}^n c_i \phi_i$ must be generic and take vector C as parameter (in order to be reused in exercise 3). The function may have the following signature:

```

1 def eSumCiphiP1(C, mesh, x):
2     # C : vector of (c_i)
3     # mesh: mesh defining the node basis
4     # x : where to valuate sum_i c_i*phi_i (in P1)
5     ...

```

- the result should be equivalent to the use of a plot style for plotting the points (x_i, c_i) , but only with P_1 .

Exercise 2 : P_2 approximation

Same exercise as exercise 1, question 2 but with P_2 elements.

Exercise 3 : Approximation errors

Indication : Integrals over the domain are always first splitted as the sum over each element $\mathcal{K}_i$.

Programming issue : We need a mesh that express the elements used in the formalism. This will be mandatory in next sessions.

Preliminary programming work : In your exercice file, write two new python classes (`MeshP1` and `MeshP2`) that derive from `Mesh` (these two class will be extended in next lab). First, we simply need to add a `NElems` and a `Elems` attributes to store the value of N and the list of element indices. Example below for P_1 :

```

1 class MeshP1(Mesh):
2     """P1 mesh"""
3     def __init__(self, a, b, N):
4         super().__init__(a, b, N) # calls the Mesh.__init__ function.
5         # Number of elements
6         self.NElems = .... # Compute N from n
7         self.Elems = ....

```

1. Plot a representation of the spatial error $|f - I_n(f)|$ for both P1 and P2 meshes and for 2 mesh sizes.
2. Write a `ErrorL2F` function in your exercice file that is taking a function and discrete values as parameters.

Indication: the function should have the following structure:

```

1 def ErrorL2F(eSumCiphi, C, funcExact, quad, mesh):
2     # eSumCiphi: function for evaluating sum_i c_i phi_i (from exercices 1 and 2)
3     # C: vector of (c_i)
4     # funcExact: exact function (f)
5     # quad: Quadrature formula from Integration.py
6     # mesh: mesh defining the nodes basis
7     ...
8     for K in mesh.Elems:
9         # Points associated to the element K
10        x = list(mesh.Points[i] for i in K)
11        ...
12        # Integral over K, using a quadrature formula
13        for p, w in zip(quad.points, quad.weights):
14            # p is the quadrature point (in [0;1])

```

```

15 |         # w is the quadrature weight
16 |         ...
17 |     return sqrt(...)

```

2. Compute an approximation of the L_2 norm of error : $\|f - I_n(f)\|_{L_2(\Omega)}$ using Left-rectangle rule for several meshes. Comment on these results.
3. Same question replacing left-rectangle rule by mid-point rule.
4. Same 2 questions for P_2 approximation. Draw a conclusion from these questions.
5. Draw a mesh convergence study for your P_1 and P_2 interpolations. Comment on the numerical integration formula used. What are the numerical orders obtained ?

Part 3 : Orthogonal projection

We denote $\pi_n(f)$ the approximation of f by orthogonal projection on P_k . Therefore, $\pi_n(f)$ is defined such that

$$(f - \pi_h(f), p)_{L_2(\Omega)} = 0, \forall p \in P_k$$

$(\cdot, \cdot)_{L_2(\Omega)}$ is the usual dot product on $L_2(\Omega)$ is denoted simply $(\cdot, \cdot)$.

As $\pi_n(f)$ is an element of P_k , we can express it in the basis $\{\phi_i\}$. Then the previous equation reduces to the linear system:

$$\sum_{i=0}^n c_i (\phi_i, \phi_j) = (f, \phi_j), \forall j = 0, \dots, n$$

We define a matrix of coefficients $A = ((\phi_j, \phi_i))_{i,j}$ and a vector $b = ((f, \phi_j))_j$. The vector unknown $C = (c_j)_j$ is the coordinates of $\pi_n(f)$ in P_k .

Several remarks can be made to simplify the assembly of the linear system.

- Vector b , require some computations of $\int_{\Omega} f(x) \phi(x) dx$. These integrals can be computed using any precise enough numerical integration formula.
- Because of compact small support of ϕ_i functions, the matrix A is sparse (*usage of sparse matrices is **not asked** in this exercise*).
- By definition of A , the matrix is symmetric.
- Integration domain can be decomposed by elements then the matrix has a block structure. $A = \sum_{i=0}^N A_i$ where A_i is the matrix of the coefficient using $(\cdot, \cdot)_{L_2(\mathcal{K}_i)}$ dot product. A_i matrices contains only a single bloc of $(k+1) \times (k+1)$ non zero coefficient. Since our mesh is homogenous, the values are identical for all matrices, they differs only from the localisation of the bloc. This point is equivalent to the change of variables that will be used in next exercices.
- Integrals for matrix coefficients **must** be computed exactly

1. Assemble the linear system for P_1 mesh.
2. Reuse the evaluation of approximation on fine mesh to plot the representation of $\pi_n(f)$ for P_1 mesh.
3. Complete the plot with a P_2 projection approximation.
4. Plot a representation of the spatial error $|f - \pi_n(f)|$ for both P_1 and P_2 meshes and for 2 mesh sizes.
5. Draw a mesh convergence for P_1 and P_2 projections.
6. Comments on the results obtained :

- a. Is the conclusions for Left-rectangle and mid-point formula are still valid with projection ?

- b. Is the formula used on matrix assembly and error estimation are the same ?
- c. How to choose the integrations formula (matrix assembly and error estimation)